

**Table 8: The estimated average number of new static analysis warnings introduced per repository per month in each SonarQube warning category pre-/post-Cursor adoption. See Table 9 for the definition of each category.**

| Category              | Pre Mean | Post Mean | Change   |
|-----------------------|----------|-----------|----------|
| Naming Conventions    | 12.15    | 33.48     | +21.32 ↑ |
| Code Hygiene          | 6.58     | 22.72     | +16.15 ↑ |
| Code Complexity       | 7.59     | 22.92     | +15.34 ↑ |
| Code Style            | 14.51    | 29.27     | +14.76 ↑ |
| Data Science          | 2.10     | 13.39     | +11.29 ↑ |
| React Patterns        | 9.24     | 18.75     | +9.52 ↑  |
| Type Safety           | 7.61     | 16.04     | +8.43 ↑  |
| CSS Issues            | 3.41     | 9.09      | +5.68 ↑  |
| OOP/Design            | 4.53     | 7.90      | +3.37 ↑  |
| Regex Issues          | 2.37     | 5.08      | +2.71 ↑  |
| Infrastructure        | 2.62     | 5.10      | +2.48 ↑  |
| Logic Error           | 7.57     | 9.66      | +2.09 ↑  |
| Security              | 1.78     | 3.76      | +1.98 ↑  |
| Empty/Incomplete Code | 5.37     | 6.98      | +1.61 ↑  |
| Error Handling        | 5.28     | 6.44      | +1.16 ↑  |
| Accessibility         | 11.99    | 12.77     | +0.78 ↑  |
| HTML Structure        | 1.89     | 1.63      | -0.26 ↓  |
| Resource Management   | 3.00     | 2.60      | -0.40 ↓  |
| Concurrency           | 5.00     | 3.05      | -1.95 ↓  |
| API Usage             | 17.77    | 13.42     | -4.35 ↓  |

## D Analysis of SonarQube Warnings

To peek into what is actually driving the increase in static analysis warnings, we collect a sample of SonarQube warnings pre- and post-Cursor adoption for the treated repositories. It is merely a convenience sample, as an architectural limitation in our SonarQube analysis pipeline prevents us from precisely collecting all warnings and tracking each warning to the version that introduced it. Thus, we only use this sample for descriptive explorations rather than for causal inference (e.g., using a DiD design as in the main paper).

In total, we collected 195,010 warnings generated from 933 SonarQube analysis rules. The first author of the paper iteratively works with Claude Opus 4.5 to generate a taxonomy of these rules with 20 categories (Table 9). Using this taxonomy, we estimate the average number of new static analysis warnings introduced per repository per month in each SonarQube warning category pre-/post-Cursor adoption (Table 8) using the available warnings and months. We only focus on newly introduced warnings in each month where data is still available as of January 2026, as SonarQube may routinely clean up resolved warnings in older project versions.

Table 8 reveals that 16 out of the 20 warning categories increased post-Cursor adoption, while only four decreased. The largest increases occurred in Naming Conventions (+21.32), Code Hygiene (+16.15), Code Complexity (+15.34), and Code Style (+14.76), all of which indicates that the adoption of Cursor—and the rapid development velocity associated with it—may lead to violation of common coding conventions, accumulation of artifacts (TODOs, commented out code, unused variables), and more complex code (e.g., deeply nested functions). We also observe increases in violation of domain-specific best practices in Data Science (+11.29), React (+9.52), Type Safety (+8.43), CSS (+5.68), etc. These violations may come from

current LLMs being trained on low-quality code or developers heavily vibe-coding and not rigorously reviewing AI-generated code, but future research is necessary to explore the true causes behind them. Interestingly, Logic Errors and Security problems—actual bugs that are both obvious enough to be detectable by static analysis and potentially dangerous—also increased modestly (+2.09 and +1.98). The few categories that decreased include API Usage (−4.35), Concurrency (−1.95), Resource Management (−0.40), and HTML Structure (−0.26), potentially indicating that AI models trained on recent code may suggest more modern API alternatives and handle certain asynchronous patterns more effectively.

Overall, while the majority of the new static analysis warnings introduced after Cursor adoption are style and maintainability issues, we also observe non-negligible increases in warning categories signaling bad coding practices (e.g., type safety) and critical bugs (e.g., security). These findings strengthen our recommendation in Section 5.2, that high-velocity AI-powered development generally introduces, rather than resolves, code quality issues, and quality assurance needs to scale with this AI-era velocity.